

Agent Failure Diagnosis — 지표 산출 명세서 및 재현 가이드

paper_final v0.2 부속 문서 · 2026-08-18 · 실측 파이프라인: normalize.py / e1_coverage.py / l3_derive.py / kappa.py / e3_judge.py

0. 산출물 상태 요약

Table A. 지표별 상태와 산출 경로

지표(논문 표)	상태	데이터	산출 프로그램	핵심 수치
Table 7 커버리지 (E1)	실측 완료	3 벤치 532 건 전수	normalize.py → e1_coverage.py	AEB 76.9 / TRAIL 61.5 / W&W 42.3% (AG 38.5·HC 53.8)
Table 8a 골드 앵커 분포	실측 완료	골드 526 건(제외 6)	normalize.py → l3_derive.py	초반 편중 48~67%, 전파 W&W-HC 평균 35.5
판정 파일럿 (E2 절)	실측 완료	9 건(골드 격리)	수기 판정 → 대조 스크립트	exact 4/9 · ±3 8/9 · 에이전트 4/4 · 모듈 1/5
Table 6 판정 안정성 (E2)	가상(미실행)	—	e3_judge.py --aggregate + kappa.py	API 실행 후 대체
Table 8b 발생률 (E3)	가상(미실행)	—	e3_judge.py --run	API 실행 후 대체
Table 9 모델 민감도	가상(미실행)	—	e3_judge.py --aggregate	API 실행 후 대체

1. 데이터 취득과 전처리 (traces.jsonl)

세 벤치마크를 아래 경로에서 취득한 뒤 normalize.py 로 공통 스키마 JSON Lines(traces.jsonl, 532 행)를 생성한다. 원본 파일은 수정하지 않는다.

Table B. 데이터 소스와 검증 결과

벤치마크	취득 경로	건수	전처리 검증(실측)
Who&When	git clone github.com/mingyin1/Agents_Failure_Attribution → Who&When/하위 JSON	AG 126 + HC 58	mistake_step 0-based 확인(min=0, 184 건 전부 0-based 적합·1-based 는 164 건만 적합). AG 는 history[].name, HC 는 role 이 에이전트 식별자
AgentErrorBench	GitHub ulab-uiuc/AgentDebug 의 안내에 따른 저자 Google Drive 폴더 → zip	ALF 100 + WS 50 + GAIA 50	라벨-궤적 ID 매칭 200/200. critical_failure_step 1-based 확인 (min=1, 199/200 적합) → 0-based 로 -1 변환. 범위 초과 1 건(GAIA_003, step 3 > n=1)은 gold.step=null 처리. 라벨 planning→plan 정규화
TRAIL	HuggingFace PatronusAI/TRAIL(게이트 동의) → data/ parquet 2 개	GAIA 117 + SWE 31	trace·labels 는 JSON 문자열, 스펠 내부는 파이썬 repr → ast.literal_eval. 스펠 트리 DFS 평탄화로 0-based 인덱스 부여. 오류 841 개 중 836 파싱, 위치 span_id 사상 실패 2 건, gold 단계 보유 143/148

실행: python3 normalize.py --whowhen <W&W 경로> --aeb <AEB 경로> --trail-gaia <gaia.parquet> --trail-swe <swe.parquet> -o traces.jsonl. 실행 로그에 서브셋별 필드 가용률(감사 근거)이 출력되며, 이것이 §2 의 E/B 판정 증빙이다.

2. 필드 가용성 플래그 (E / B / 부재)

trace 마다 18 개 필드 플래그를 부여한다. E(explicit): 스키마상 분리 기록된 구조 필드. B(embedded): 별도 필드는 없으나 자유텍스트 안에 판정 근거가 존재 — 아래의 사전 선언된 정규식 패턴으로 검출한다. 부재: 둘 다 아님. 판정 방식 R 항목은 E 를 요구하고, L 항목은 B 를 허용하며, R+L 은 R 성분 필드에 E 를 요구한다(요구 수준 규칙).

Table C. 매물 근거(B) 검출 패턴 (normalize.py 원문 그대로)

필드	패턴(대소문자 무시)
plan_field	\b(initial plan plan: we will steps?: next step 계획)\b
action_field	(\x60\x60\x60 \x60w+\\(-> s*\w+ perform(ed)? the (following)?(search action) click type[d]? navigate execute)
observation	(output: result: returned viewport screenshot https?: / I found according to the page)
reflection_field	\b(verify check(ed)ing)? however it seems not (yet)?(complete correct) termination condition 재검 확인)\b
error_msg	\b(error exception traceback failed to invalid)\b · AEB 는 환경 고유 마커("Nothing happens"=ALFWorld, "Invalid action"=WebShop) 존재 시 E 로 승격
memory_field	단계 수 > 1 (이전 이력 자체가 회상 판정의 근거)

민감도 경고: B 판정은 패턴 선택에 민감하다. 실측에서 W&W-AG 의 plan 가용률이 53.2%로 임계(90%) 미달 → L2-P 군 전체 탈락. 패턴·임계 변경 시 커버리지가 이동하므로, 논문에는 패턴 원문과 임계 p 민감도를 병기한다.

3. E1 — 진단가능성 커버리지 (Table 7)

3.1 산출식

퀘적 j 에서 항목 i 가 "판정가능" [$D_i(j)=1$] $\Leftrightarrow$ 항목 i 의 요구 필드가 전부 요구 수준(E 또는 E|B)으로 존재. 파생 항목(L3-02·03·05)은 입력 항목이 판정가능하고 step_index 가 E 일 때 1. L3-01 은 L2 항목 중 1 개 이상 판정가능일 때 1.

벤치마크 수준: $p_i = (1/n) \sum_j D_i(j)$. 항목 판정가능 $\Leftrightarrow p_i \geq 0.9$. N/A(개념 원천 부재)는 분모 제외 — 본 실측에서 N/A = 0.

커버리지 = $|\{ i : p_i \geq 0.9 \}| / (26 - |N/A|)$.

3.2 항목별 요구 필드 (26 항목 전체)

Table D. 항목 → 요구 필드(요구 수준) → 파생 의존

항목	방식	요구 필드(수준: E=구조 필수, EB=매물 허용)	파생 의존
L0-01	R	task_spec(E)	
L0-02	R	step_index(E)	
L0-03	R	module_tags(E)	
L0-04	R	observation(E)	
L0-05	R	outcome_label(E); outcome_rationale(E)	
L0-06	R	raw_tool_response(E)	
L0-07	R	sys_metadata(E)	
L1-01	R	step_index(E); step_limit_cfg(E)	
L1-02	R	action_field(E); error_msg(E)	
L1-03	R+L	observation(E); error_msg(EB)	
L1-04	R	action_field(E)	
L2-M1	L	memory_field(EB); step_index(EB)	
L2-M2	L	memory_field(EB); step_index(EB)	
L2-M3	L	memory_field(EB); observation(EB)	
L2-R1	L	reflection_field(EB); outcome_label(EB)	
L2-R2	L	reflection_field(EB); observation(EB)	
L2-P1	L	plan_field(EB); task_spec(EB)	
L2-P2	L	plan_field(EB); observation(EB)	
L2-P3	L	plan_field(EB)	
L2-A1	L	plan_field(EB); action_field(EB)	
L2-A2	R+L	action_field(E); observation(EB)	

L3-01	L	(파생 전용)	L2 중 1 개 이상
L3-02	Rd	step_index(E)	L3-01
L3-03	Rd	step_index(E)	L3-01
L3-04	L	reflection_field(EB); plan_field(EB)	
L3-05	Rd	(파생 전용)	L2 중 1 개 이상

3.3 실행과 재현

```
python3 e1_coverage.py traces.jsonl --p 0.9 -o table7.json
```

출력: 서브셋별 {판정가능 항목 목록, 판정불가 항목 목록, 커버리지}. 실측값 — AEB-ALF/WS 20/26=76.9%, AEB-GAIA 13/26=50.0%(memory·reflection 태고 88%로 임계 미달), TRAIL-GAIA/SWE 16/26=61.5%, W&W-AG 10/26=38.5%, W&W-HC 14/26=53.8%. 임계 민감도는 --p 0.8/0.85 로 재실행하여 얻는다(예: p=0.85 면 AEB-GAIA 가 76.9%로 복귀).

4. 골드 앵커 분포 (Table 8a)

4.1 정의 (0-based 통일)

T = 궤적 단계 수, t_0 = 골드 주석의 결정적(최초) 오류 단계. TRAIL 처럼 오류가 다수(궤적당 평균 5.7 건)면 스펠 위치를 DFS 인덱스로 사상한 뒤 최솟값을 t_0 로 채택.

위치 = $t_0/(T-1)$ 의 3 분위: [0, 1/3) 초반 / [1/3, 2/3) 중반 / [2/3, 1] 종반 (T=1 이면 초반). 전파 길이 = (T-1) - t_0 . 제외 규칙: gold.step=null(AEB 범위 초과 1 건, TRAIL 사상 실패·오류 0 건 5 건)은 분포에서 제외하고 각주로 보고.

4.2 실행과 실측값

```
python3 l3_derive.py traces.jsonl -o table8_gold.json
```

실측 — W&W 전체(184): 초반 48.4%, 전파 평균 14.4/중앙값 5 (HC 는 평균 35.5, 최대 124). AEB(199): 초반 66.8%, 전파 평균 17.2/중앙값 22. TRAIL(143): 중반 48.3% 편중, 전파 평균 19.0/중앙값 10.

5. 일치도 지표 ($\kappa \cdot AC1$)

Cohen's $\kappa = (Po - Pe)/(1 - Pe)$, Po = 일치율, $Pe = \sum_l p_1(l) \cdot p_2(l)$ (평가자별 라벨 주변 분포의 곱). Gwet AC1 = $(Po - Pe')/(1 - Pe')$, $Pe' = \sum_l \pi(l)(1-\pi(l))/(q-1)$, $\pi(l)$ = 두 평가자 평균 라벨 비율, q = 라벨 수. 발생률이 한쪽으로 쏠린 항목에서 κ 가 붕괴하는 κ 역설을 AC1 이 완충한다.

```
python3 kappa.py --selftest # 검증 3 종: 2x2 예제  $\kappa=0.4000$  / 완전일치  $\kappa=1$  /  $\kappa$  역설 ( $Po=0.95$ ,  $\kappa=-0.02$ ,  $AC1=0.95$ )
```

```
python3 kappa.py sheet.json # [{"id":..., "r1":..., "r2":...}] 형식
```

6. 판정 파일럿 (E2 절 실측 근거)

프로토콜: (i) 표본 — 골드 단계가 존재하고 단계 수 ≥ 2 (GAIA 는 ≥ 4)인 궤적 중 최단 순으로 W&W-AG 4 건 + AEB-GAIA 3 건 + AEB-ALFWorld 2 건 = 9 건. (ii) 격리 — 궤적 본문(pilot_traces.md)과 골드(pilot_gold.json)를 분리 저장하고, 심사자는 판정 완료 후에만 골드를 연다. (iii) 판정 — 궤적당 최초 결정적 오류 단계·모듈(또는 에이전트)·해당 L2 항목·확신도를 기록(pilot_judgments.json). (iv) 대조 — 단계는 |판정-골드| $\leq k$ ($k=0,1,3$), 모듈·에이전트는 명칭 일치.

실측 결과: 단계 exact 4/9, ± 1 5/9, ± 3 8/9 (± 3 밖 1 건은 ALFWorld 의 계획 오개념 대 골드의 memory 과단순화 관점 차). W&W 오류 에이전트 4/4 일치. AEB 오류 모듈 1/5 — 골드는 memory/over_simplification 집중, 심사자는 근본 원인 모듈을 지목하는 체계적 관점 차. 함의: 골드 앵커 정합은 단계($\pm k$) 중심으로 보고, 모듈 일치는 보조 지표.

7. E3·E2 — LLM 판정 (미실행, 실행 절차)

7.1 파이프라인 사양 (e3_judge.py 에 구현 완료)

Table E. E3 실행 사양

항목	사양
판정 모델	상위 claude-sonnet-4-6 / 저가 claude-haiku-4-5-20251001, temperature 0
프롬프트 분할	모듈군 5 개: M(L2-M1~3) / R(L2-R1~2) / P(L2-P1~3) / A(L2-A1~2) / T(L3-01~04) → 퀘적당 5 콜
출력 강제	JSON {items:{ID:{v:true false null, step, why}}}, 스키마 위반 시 1 회 재시도 후 실패 기록
절단 정책	스텝당 1,200 자, 퀘적당 100,000 자(초과 시 앞 60%·뒤 40% 보존, 생략 표기)
오염 방지	골드 주석은 프롬프트에 절대 포함하지 않음(코드 수준에서 gold 필드 미사용)
변형 b	문항 순서 역전 + 지시문 재서술 — 상위 모델 × 20% 증화 표본(시드 20260817)
재개	출력 jsonl 의 (trace_id, group) 기준 스킵 — 중단 후 같은 명령 재실행 시 이어서, 중복 과금 없음

7.2 비용 산식과 견적 (실측 프롬프트 기준)

토큰 $\approx$ 문자수/4 (여유율 1.2 병기). 실측 프롬프트 총량: 입력 14.53M tok(여유 17.44M), 출력 = 2,660 콜 $\times$ 700 tok = 1.86M tok/모델. 비용 = 입력 tok $\times$ 단가_{in} + 출력 tok $\times$ 단가_{out}. 2026-08 표준 단가(Sonnet \$3/\$15, Haiku \$1/\$5) 기준: Sonnet \$80.24 + Haiku \$26.75 + 변형 표본 $\approx$ \$21 $\rightarrow$ 표준 약 \$128, 배치 API(50% 할인) 약 \$64. 단가는 실행 전 콘솔에서 재확인.

7.3 실행 명령 (전체 시퀀스)

```
export ANTHROPIC_API_KEY=sk-ant-...
python3 e3_judge.py traces.jsonl --run --model sonnet --limit 20 --out test.jsonl # 소액 점검
python3 e3_judge.py traces.jsonl --run --model sonnet --out judg_sonnet.jsonl
python3 e3_judge.py traces.jsonl --run --model haiku --out judg_haiku.jsonl
python3 e3_judge.py traces.jsonl --run --model sonnet --variant b --limit 106 --out
judg_vb.jsonl
python3 e3_judge.py traces.jsonl --aggregate judg_sonnet.jsonl judg_haiku.jsonl judg_vb.jsonl
```

마지막 명령이 항목별 발생률(Table 8b), 모델 간 κ -AC1(Table 6), 골드 앵커 L3-01 정합 exact/ $\pm 1/\pm 3/\pm 5$ (Table 9 기준점)를 출력한다. 출력값으로 논문의 가상 표 3 개를 대체하면 개정이 완료된다.

8. 환경·파일 목록

Python 3.12(+pyarrow, TRAIL parquet 용), API 실행은 표준 라이브러리만 사용(외부 SDK 불필요). 난수 시드 20260817. 산출 파일: traces.jsonl(공통 스키마 532 행) · table7.json · table8_gold.json · e1_report.txt · l3_report.txt · pilot_traces.md / pilot_gold.json / pilot_judgments.json · (실행 후) judg_*.jsonl. 전 스크립트는 afd_full_pipeline.zip 으로 배포.